FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 7. Herramientas de Git

Selección de revisiones, preparación interactiva, historia, depuración y submódulos

Más allá del flujo básico, Git ofrece herramientas avanzadas para localizar commits, preparar cambios con precisión, reescribir la historia y depurar errores. Este capítulo resume las más útiles.

7.1 Selección de revisión

- **Hashes abreviados:** bastan los primeros 7 caracteres si no son ambiguos.
- **Referencias relativas:** `HEAD^` es el padre del último commit, `HEAD~3` retrocede tres.
- **Rangos:** `git log main..tema` muestra lo que está en `tema` y no en `main`; `A...B` muestra lo que está en uno solo de los dos.
- **Reflog:** `git reflog` registra a dónde apuntó HEAD y permite recuperar commits que parecían perdidos.

```
git show a1b2c3d
git log HEAD~3..HEAD
git reflog
```

7.2 Preparación interactiva

Permite confirmar solo parte de los cambios de un archivo, para crear commits atómicos:

```
git add -p        # elegir fragmentos (hunks) a preparar
git add -i        # menú interactivo
```

7.3 Guardado rápido (stash) y limpieza

`git stash` guarda temporalmente el trabajo sin confirmar para cambiar de rama y recuperarlo después. `git clean` elimina archivos no rastreados.

```
git stash
git stash list
git stash pop
git clean -n      # simulacro, no borra nada
git clean -fd     # borra archivos y directorios sin rastrear
```

7.4 Firmar y buscar

Los commits y etiquetas pueden firmarse con GPG (`git commit -S`, `git tag -s`) para verificar su autoría. Para buscar se usan:

```
git grep -n "palabra"        # busca en el árbol de trabajo
git log -S"función"          # commits que añadieron o quitaron ese texto
git log -L :funcion:archivo.py # historia de una función
```

7.5 Reescribir la historia

Antes de publicar, es posible ordenar los commits: modificar el último (`--amend`), reordenar, unir (*squash*) o dividir commits con un rebase interactivo. Todo esto cambia los hashes, por lo que **nunca debe hacerse sobre commits ya compartidos**.

```
git commit --amend
git rebase -i HEAD~3   # pick, reword, edit, squash, fixup, drop
```

7.6 Reset desmitificado

`git reset` mueve la rama actual y, según la opción, actualiza el área de preparación y el directorio de trabajo:

- `--soft`: solo mueve HEAD; los cambios quedan preparados.
- `--mixed` (por defecto): además vacía el área de preparación; los cambios quedan en el directorio de trabajo.
- `--hard`: también reescribe el directorio de trabajo; **destruye cambios sin confirmar**.

```
git reset --soft HEAD~1
git reset --hard HEAD~1
```

7.7 Fusión avanzada, rerere y depuración

Ante conflictos complejos se puede abortar la fusión con `git merge --abort` o rehacerla con otra estrategia. `git rerere` recuerda cómo se resolvió un conflicto para aplicarlo automáticamente si reaparece. Para depurar:

```
git blame archivo.py        # quién modificó cada línea
git bisect start
git bisect bad               # el commit actual tiene el error
git bisect good v1.0         # esta versión funcionaba
git bisect reset
```

7.8 Submódulos y otras utilidades

Los submódulos permiten incluir un repositorio dentro de otro, fijando una versión concreta. `git bundle` empaqueta un repositorio en un único archivo para transferirlo sin red. Los asistentes de credenciales (`credential.helper`) evitan escribir la contraseña o el token en cada operación.

```
git submodule add https://github.com/usuario/lib.git libs/lib
git clone --recurse-submodules <url>
git submodule update --init --recursive
git config --global credential.helper cache
```

Conclusión del capítulo

Estas herramientas no se usan a diario, pero marcan la diferencia al resolver problemas: recuperar trabajo perdido, encontrar el origen de un error o dejar una historia clara antes de compartirla.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2